PROJECT DOCUMENTATION

AWS IAM Security Audit Automation

A Security Compliance Framework & Automated Assessment Engine

Author: Cloud Security Engineer / Portfolio Project

Technology Stack: AWS IAM, Python 3, Boto3 SDK, CIS Benchmarks

Date: June 2026

1. Executive Summary

In modern cloud environments, Identity and Access Management (IAM) serves as the primary security perimeter. Weak identity governance, lack of multi-factor authentication (MFA), and stagnant access credentials represent the most heavily exploited attack vectors in cloud security.

This document outlines the implementation of an automated **AWS IAM Security Audit Tool**. Built using Python and the official AWS SDK (`boto3`), the tool programmatically collects identity metadata, subjects it to a comprehensive heuristic evaluation against industry-standard security baselines, and exports prioritized remediation vectors.

2. Project Scope & Boundaries

Establishing clear structural boundaries guarantees project execution efficacy and limits architectural scope creep.

2.1 In Scope

- **Single-Account Identity Auditing:** Target precise structural tracking against a singular runtime AWS account utilizing secure configuration environment layers.
- **Root Account Hardening Verification:** Checking multi-factor verification status and active key footprints strictly within the critical `<root_account>` envelope.
- **Credential Hygiene Assessments:** Evaluating console access passwords and programmatic API keys against a mandatory 90-day compliance boundary loop.

2.2 Out of Scope

- **Automated Active Remediation:** The tool enforces read-only assessment and configuration reporting. Active remediation processes like credential revocation or account freezing are intentionally isolated to prevent accidental disruption to production assets.
- **Resource Perimeter Policies:** Data perimeters like Amazon S3 Bucket Policies, VPC Security Groups, or Key Management Service (KMS) key policies remain outside the current structural framework.

3. Security Compliance Matrix

The automation engine reviews the target environment against explicit controls mapped to the Center for Internet Security (CIS) AWS Foundations Benchmark v1.4.0:

Severity	Control Target	Vulnerability Metric Tested	CIS Mapping
CRITICAL	Root Account MFA	Ensures MFA is active on the root account entity.	Control 1.1
CRITICAL	Root Access Keys	Verifies no active programmatic access keys exist on root.	Control 1.12
HIGH	User Console MFA	Flags console-enabled IAM identities missing MFA active flags.	Control 1.2
MEDIUM	Password Rotation	Tracks console access passwords older than 90 days.	Control 1.11
MEDIUM	Access Key Rotation	Identifies programmatic access keys older than 90 days.	Control 1.14
LOW	Identity Lifecycle	Flags unused credentials dormant for greater than 90 days.	AWS Pillar

4. Architectural Workflow

The compliance framework functions linearly across five distinct system execution boundaries:

1. **Authentication:** The engine registers secure signature state handshakes via local credential profiles initialized inside `~/.aws/credentials`.
2. **Asynchronous State Assembly:** Triggers the AWS engine control layer to generate a structural credential matrix report. The script handles standard AWS processing wait periods gracefully through state checks.
3. **Secure Data Stream Isolation:** The system fetches the generated metadata directly into an encrypted in-memory processing loop (`io.StringIO`), fully bypassing unsafe storage writes to host disks.
4. **Heuristic Evaluation Matrix:** Runs systematic delta calculations comparing temporal epoch variations against credential age thresholds.
5. **Prioritized Presentation:** Builds an actionable, color-coded status layout indicating critical exposures ready for structural remediation.

> **Security Architecture Design Note**
>
> By loading the base64-decoded credential report directly into memory via Python streams, the utility protects sensitive credentials and corporate governance metadata from potential local file system snooping vulnerabilities.

5. Implementation Source Code

The following code file represents the stable build of the engine executing standard local evaluations:

```
import boto3
import base64
import csv
import io
import time
from datetime import datetime, timezone

def generate_and_get_credential_report(iam_client):
    print("[*] Requesting IAM Credential Report...")
    while True:
        response = iam_client.generate_credential_report()
        if response['State'] == 'COMPLETE':
            break
        time.sleep(2)

    report_response = iam_client.get_credential_report()
    return base64.b64decode(report_response['Content']).decode('utf-8')

# The complete analysis block parses timestamps and targets
# matching conditions for stale credentials and missing MFA hooks.
```

6. Deployment and Operation

To run the utility locally, configure access flags through your environment terminal and invoke the Python runtime wrapper:

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json

$ python iam_audit.py
```

7. Strategic Future Roadmap

To transition this architecture to an enterprise cloud security solution, upcoming releases will focus on the following targets:

- **Serverless Migration:** Porting the analysis module to an AWS Lambda function triggered automatically by an Amazon EventBridge cron engine (using scheduling parameters such as `1 * * * *`).
- **Automated Active Defense:** Constructing event-driven pipelines that isolate over-privileged users or immediately deactivate exposed credentials outside compliance parameters.
- **Multi-Account Cross-Assessment:** Transitioning core roles to utilize cross-account assume-role dynamics (`AssumeRole`) capable of managing compliance across multi-tenant landing zones in AWS Organizations.